Customer Shopping Behaviour Analysis

1. Project Overview

This project analyses customer shopping behaviour using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behaviour to guide strategic business decisions.

2. Dataset Summary

- Rows: 3,900
- Columns: 18
- Key Features:
- Customer demographics (Age, Gender, Location, Subscription Status)
- Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
- Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

● Data Loading: Imported the dataset using pandas.

● Initial Exploration: Used df.info() to check structure and df.describe(include = 'all') for summary statistics.

In 2
```
import pandas as pd
df = pd.read_csv('customer_shopping_behavior.csv')
```

In 3
```
df.head()
```

Out 3

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise

5 rows × 18 columns Open in new tab

```
In 5   df.describe(include = 'all')
```

Out 5

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN

11 rows × 18 columns Open in new tab

- **Missing Data Handling**: Checked for null values and imputed missing values in the Review Rating column using the median rating of each product category.

```
In 6   df.isnull().sum()
```

```
Out 6
Customer ID              0
Age                      0
Gender                   0
Item Purchased           0
Category                 0
Purchase Amount (USD)    0
Location                 0
Size                     0
Color                    0
Season                   0
Review Rating           37
```

```
In 7   df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

- **Column Standardization**: Renamed columns to snake case for better readability and documentation.
- **Feature Engineering:**

○ Created age_group column by binning customer ages.

○ Created purchase_frequency_days column from purchase data.

```
In 9
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ','_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})
```

```
In 10   df.columns
```

```
Out 10
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

```
In 11
# create a column age_group
labels = ['Young Adult', 'Adult', 'Middle-aged','Senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels=labels)
```

```
In 12   df[['age','age_group']].head(10)
```

- **Data Consistency Check:** Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

In 16
```python
df[['discount_applied','promo_code_used']].head(10)
```

Out 16

	discount_applied	promo_code_used
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

10 rows × 2 columns Open in new tab

In 17
```python
(df['discount_applied'] == df['promo_code_used']).all()
```

Out 17
```
np.True_
```

In 18
```python
df = df.drop('promo_code_used', axis = 1)
```

In 19
```python
df.columns
```

● **Database Integration**: Connected Python script to PostgreSQL and loaded the cleaned Data Frame into the database for SQL analysis.

In 21
```python
from sqlalchemy import create_engine

# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details
username = "postgres"
password = "******"
host = "localhost"
port = "5432"
database = "customer_behavior"

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

# Step 2: Load DataFrame into PostgreSQL
table_name = "customer"
df.to_sql(table_name, engine, if_exists="replace",index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")
```

```
Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

4. Data Analysis using SQL (Business Transactions)

We performed structured analysis in PostgreSQL to answer key business questions:

- **Revenue by Gender** – Compared total revenue generated by male vs. female customers.

	gender text	revenue numeric
1	Female	75191
2	Male	157890

- **High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62

Total rows: 839 Query complete 00:00:00.143

- **Top 5 Products by Rating** – Found products with the highest average review ratings.

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

- **Shipping Type Comparison** – Compared average purchase amounts between Standard and Express shipping.

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

- **Subscribers vs. Non-Subscribers** – Compared average spend and total revenue across subscription status.

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

- **Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.00
3	Coat	49.00
4	Sweater	48.00
5	Pants	47.00

- **Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

	customer_segment text	Number of Customers bigint
1	Loyal	3116
2	New	83
3	Returning	701

- **Top 3 Products per Category** – Listed the most purchased products within each category.

	item_rank bigint	category text	item_purchased text	total_orders bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

- **Repeat Buyers & Subscriptions** – Checked whether customers with >5 purchases are more likely to subscribe.

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

- **Revenue by Age Group** – Calculated total revenue contribution of each age group.

	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

5. Dashboard in Power BI Finally, we built an interactive dashboard in Power BI to present insights visually.

6. Business Recommendations

- **Boost Subscriptions** – Promote exclusive benefits for subscribers.
- **Customer Loyalty Programs** – Reward repeat buyers to move them into the "Loyal" segment.
- **Review Discount Policy** – Balance sales boosts with margin control.
- **Product Positioning** – Highlight top-rated and best-selling products in campaigns.
- **Targeted Marketing** – Focus efforts on high-revenue age groups and express-shipping users.